Filière : Cyber sécurité et Intelligence Artificielle

Module : Architecture Logicielle

Nom et Prénom : Abderrazak MOSTAID

Rapport : TP°4 - Architecture Orientée Services (SOA)

Objectifs

- Comprendre les principes de l'architecture SOA
- Créer des microservices avec Spring Boot
- Implémenter la communication inter-services via REST
- Gérer une base de données avec Spring Data JPA
- Utiliser un service de découverte (Eureka)

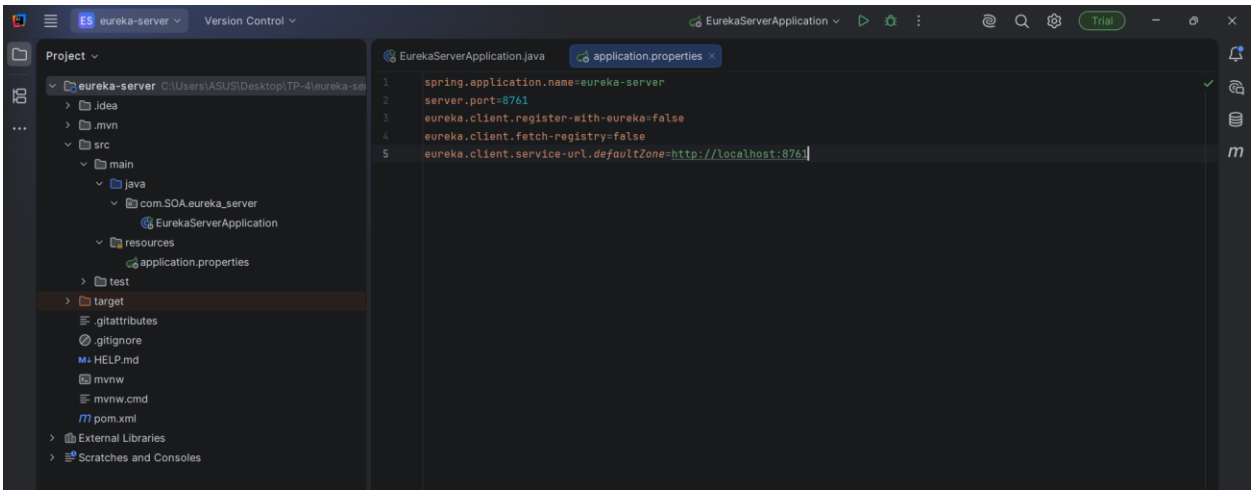
Pratique

Concernant la partie pratique on va faire une application simple base sur trois services, **product service** – **order service** – **eureka service.**

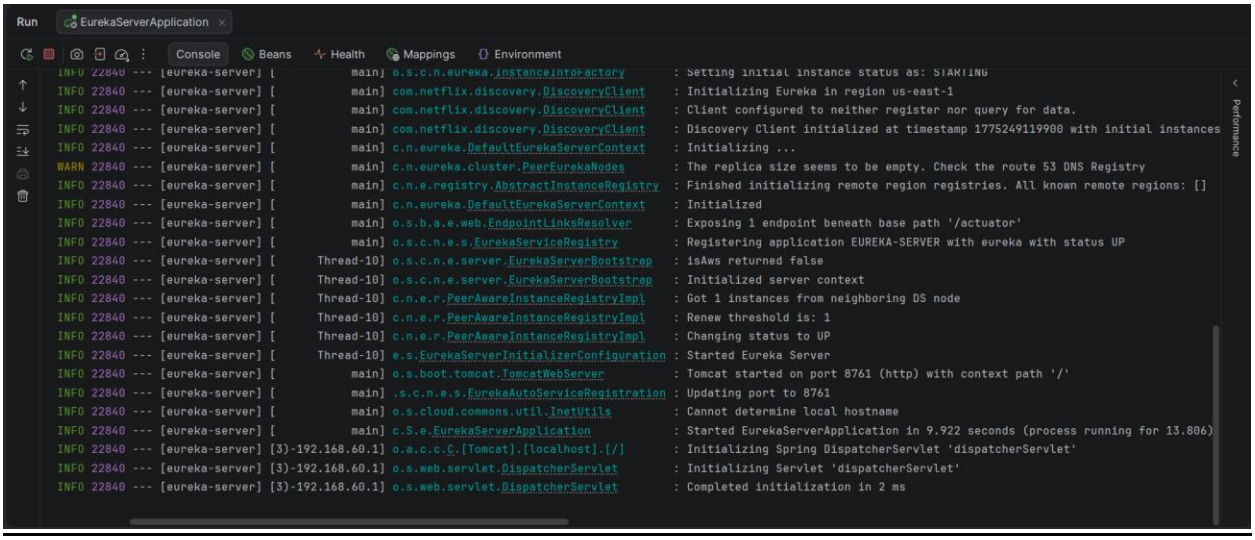
Ces trois services vont être connectées entre eux et pour le service eureka c`est pour l`enregistrement et le Heath check des autres services.

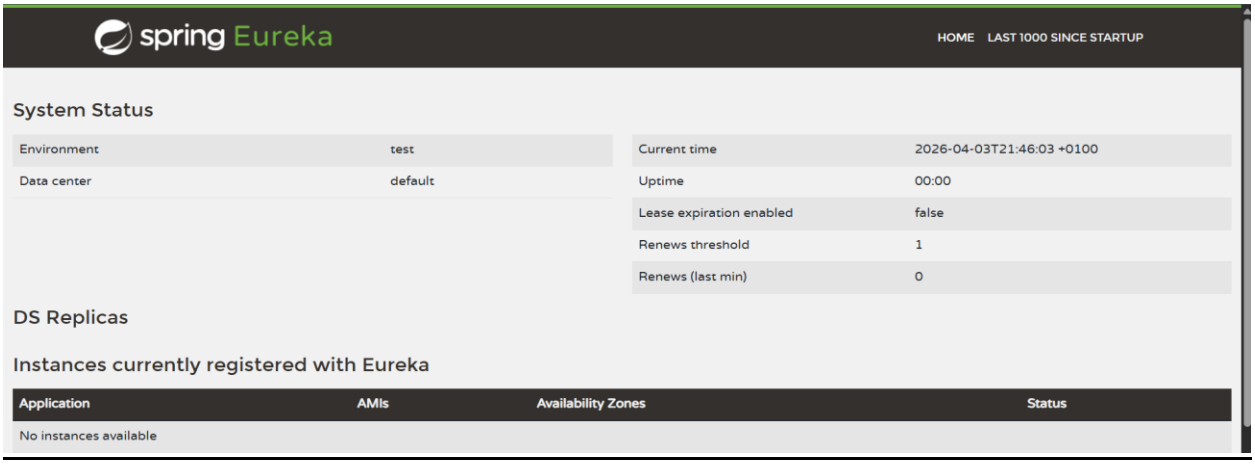
Premièrement on va commencer par le service eureka et après on va configurer les autres services.

Service-eureka

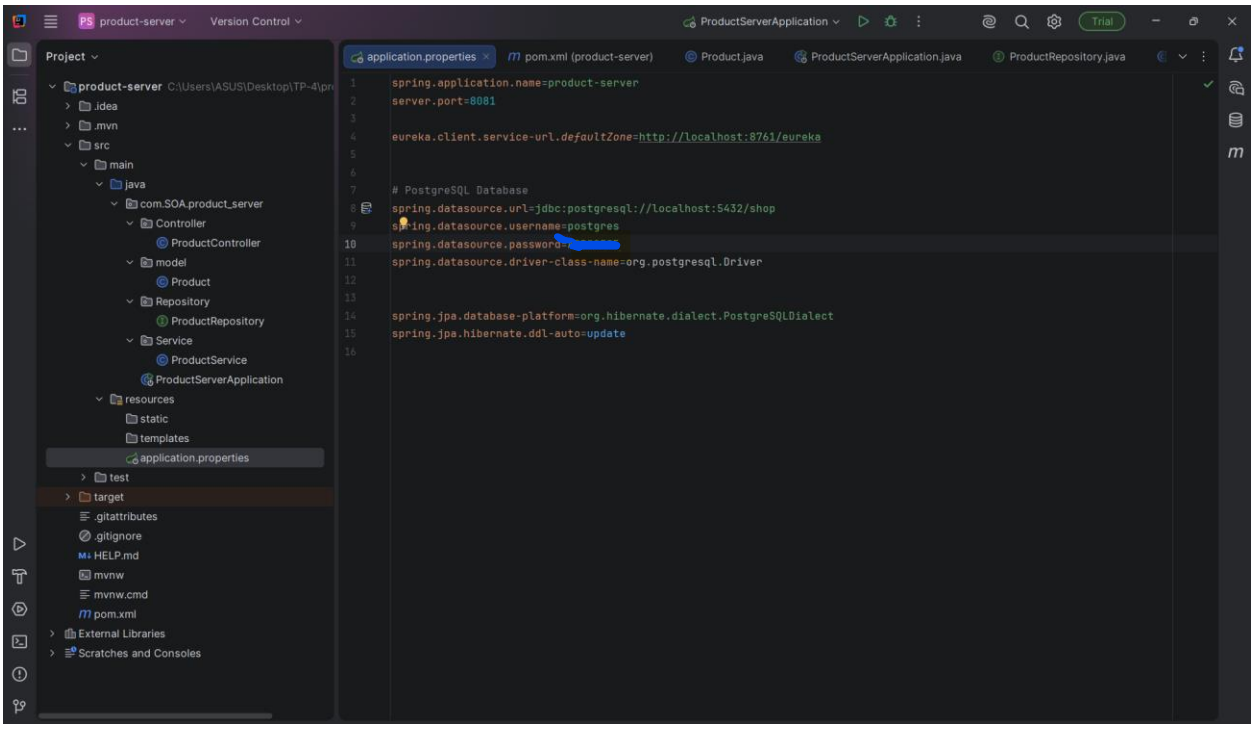


Exécution du code

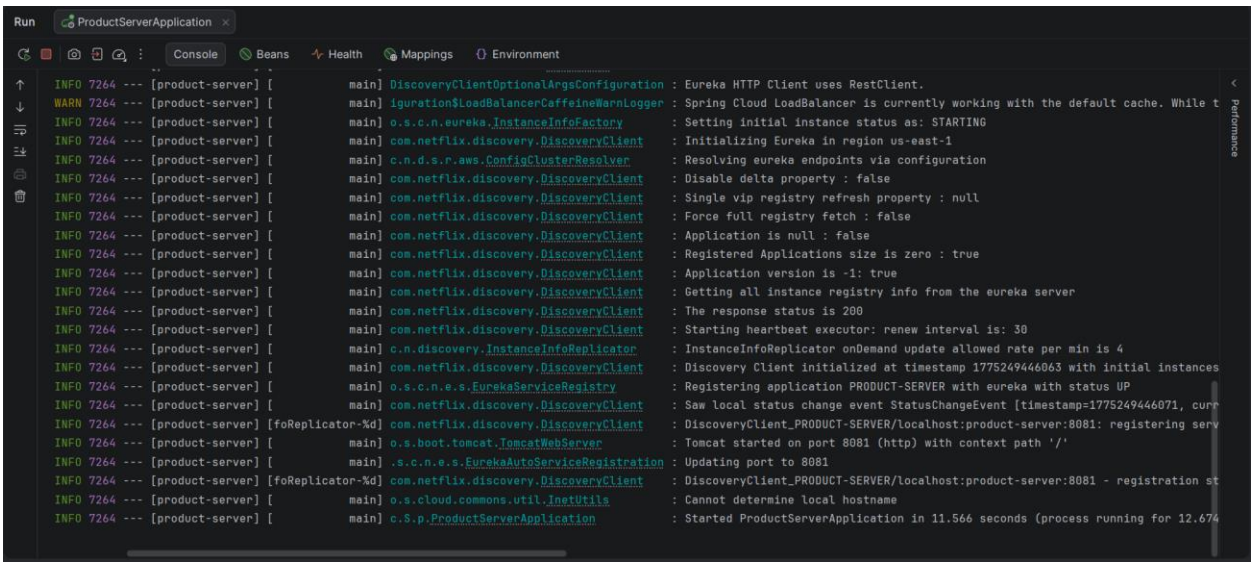




Service produit

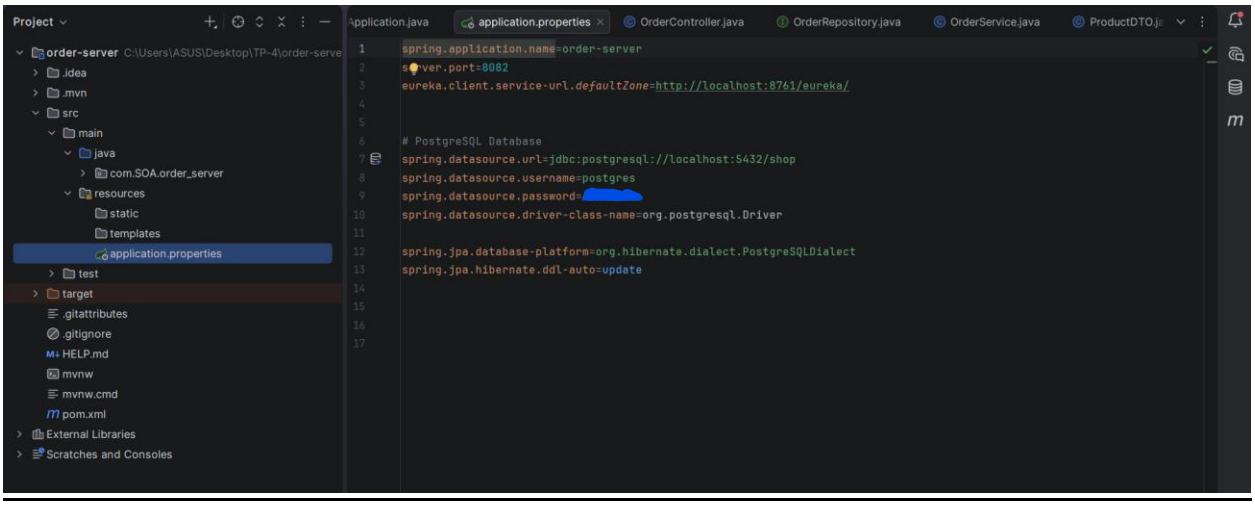


Exécution du code

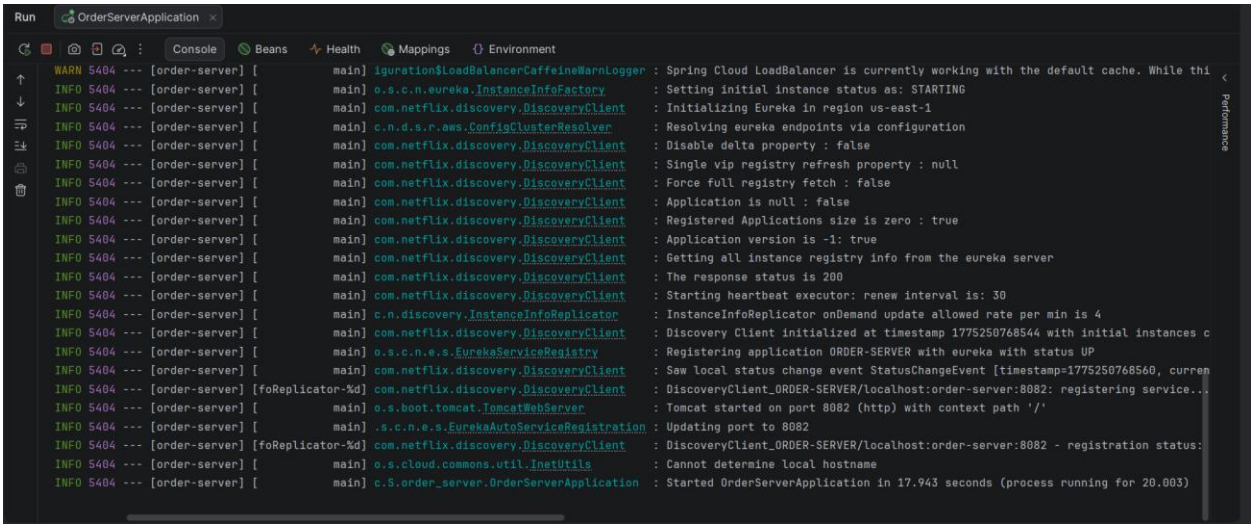


Jusqu`a maintenant on a implémenté deux services avec succès, il ne reste qu’avoir le service order.

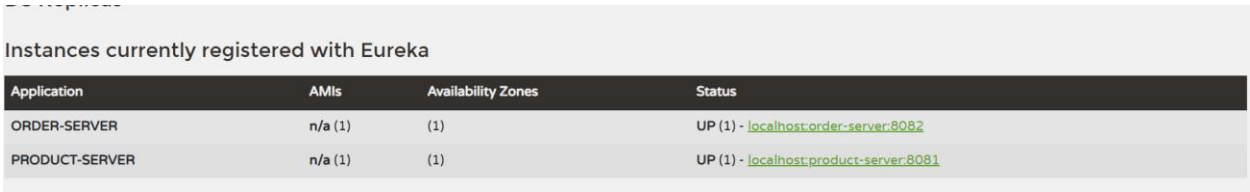
Order service



Exécution du code



Maintenant il ne reste que vérifier Dans le service eureka si les autres services apparaissent, pour l`enregistrement et le Health check.

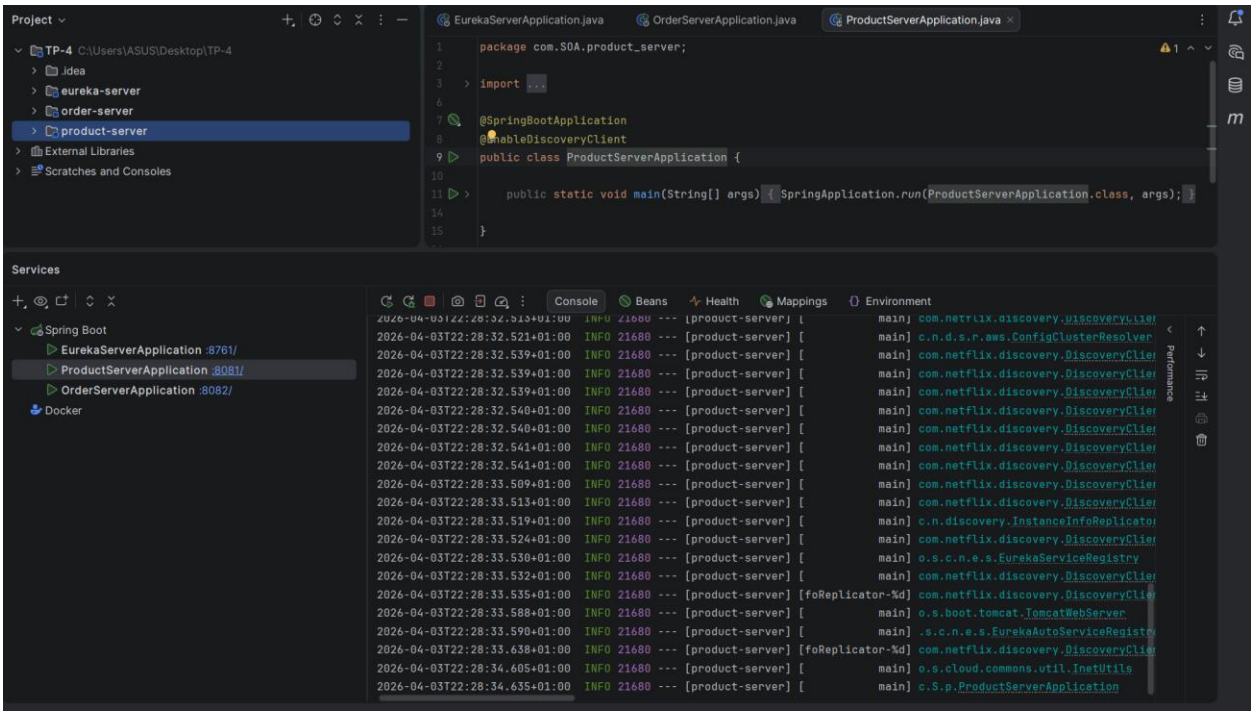


Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVER	n/a (1)	(1)	UP (1) - localhost:order-server:8082
PRODUCT-SERVER	n/a (1)	(1)	UP (1) - localhost:product-server:8081

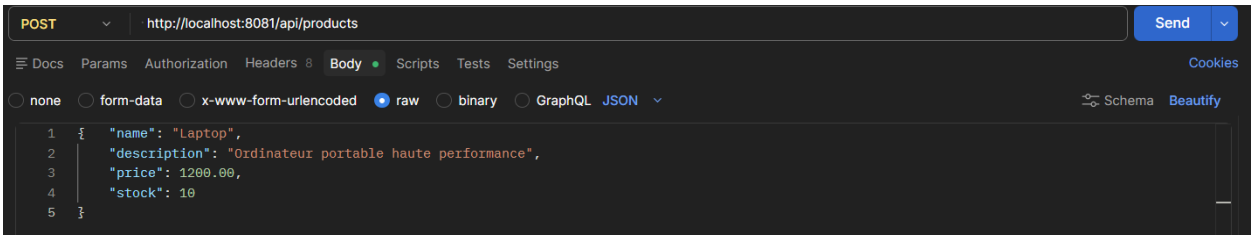
Alors les deux services sont bien présente sur le service eureka, maintenant il ne reste que tester ales requêtes avec Postman.

Structure générale de l`application



Tests avec Postman

Test 1 : Créer un produit



Test 2: Lister les produits

GET http://localhost:8081/api/products Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 5 Test Results 200 OK 474 ms 263 B

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Laptop",
        "description": "Ordinateur portable haute performance",
        "price": 1200.0,
        "stock": 10
    }
]
```


Test 3: Créer une commande

http://localhost:8082/api/orders Save Share

POST http://localhost:8082/api/orders?productId=1&quantity=2 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 5 Test Results 200 OK 1.13 s 296 B

{} JSON Preview Visualize


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop",
    "quantity": 2,
    "totalPrice": 2400.0,
    "orderDate": "2026-04-03T22:40:32.7379394",
    "status": "PENDING"
}
```


Test 4: Lister les commandes

GET http://localhost:8082/api/orders Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Body Cookies Headers 5 Test Results 200 OK 346 ms 297 B

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 2,
        "totalPrice": 2400.0,
        "orderDate": "2026-04-03T22:40:32.737939",
        "status": "PENDING"
    }
]
```

Exercice 1 : Ajouter une fonctionnalité

1 –

```java
    public Product updateStock(Long id, Integer quantity) {  no usages
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Produit introuvable avec l'id : " + id));

        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant pour le produit : " + product.getName());
        }

        product.setStock(product.getStock() - quantity);
        return productRepository.save(product);
    }
}
```

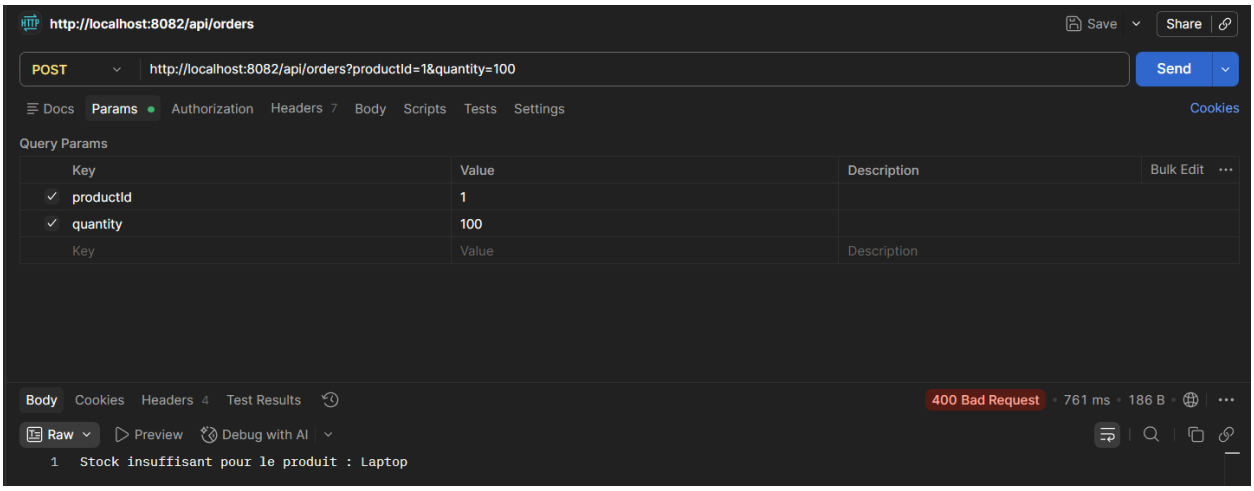
2 –

```java
@FeignClient(name = "product-server")  2 usages
public interface ProductClient {
    @GetMapping("/api/products/{id}") ProductDTO getProductById(@PathVariable Long id);
    @PutMapping("/api/products/{id}/stock")
    ProductDTO updateStock(@PathVariable("id") Long id, @RequestParam("quantity") Integer quantity);
}
```

3 –

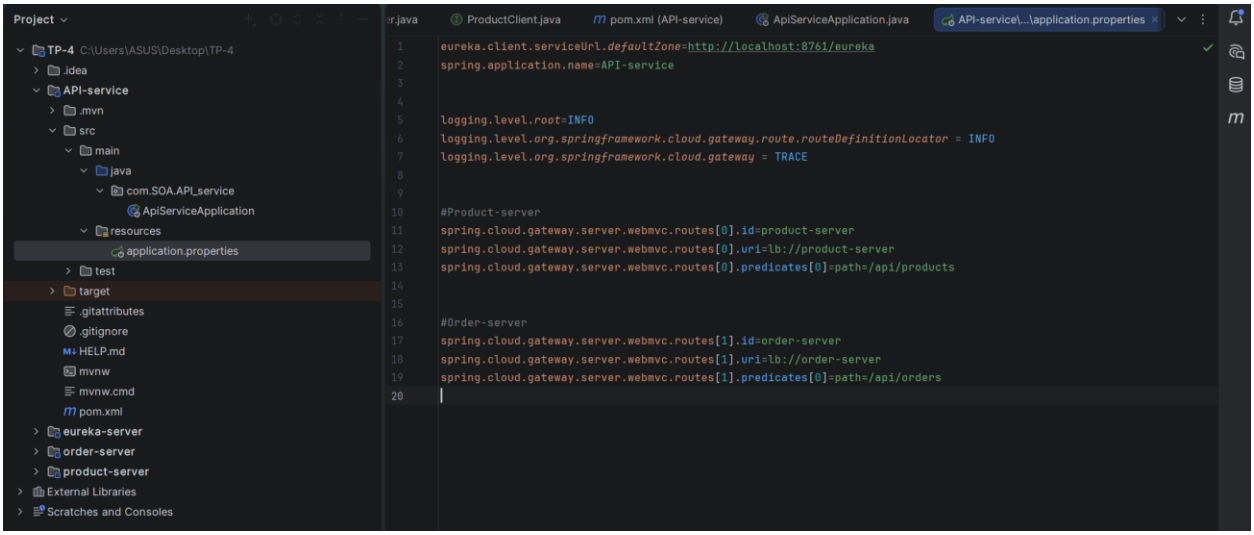
```java
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductClient productClient;
    public List < Order > getAllOrders() {  1 usage
        return orderRepository.findAll();
    }
    public Order createOrder(Long productId, Integer quantity) {  1 usage

        ProductDTO product = productClient.getProductById(productId);
        if (product == null) {
            throw new RuntimeException("Produit introuvable");
        }

        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant pour le produit : " + product.getName());
        }
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        Order savedOrder = orderRepository.save(order);

        productClient.updateStock(productId, quantity);

        return savedOrder;
    }
}
```

La gestion d`erreurs (Stock insuffisant)

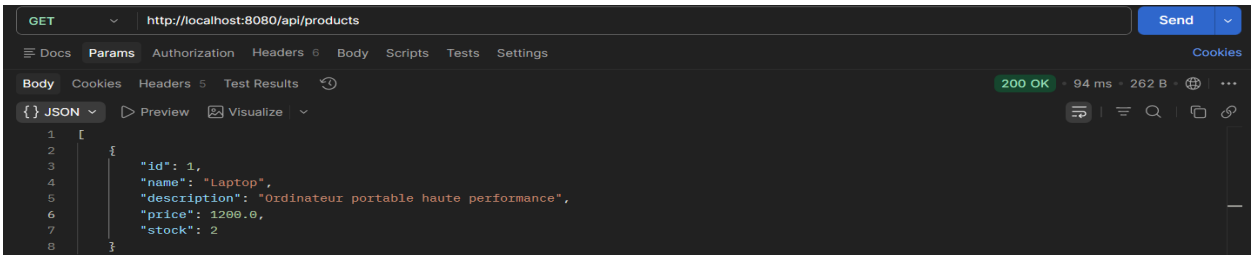


Exercice 2: Créer une API Gateway

1 – Gateway config service



2 – Verification via Potman(/api/products)



3 – Verification (/api/orders)

GET http://localhost:8080/api/orders Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Body Cookies Headers 5 Test Results 200 OK 31 ms 575 B

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 2,
        "totalPrice": 2400.0,
        "orderDate": "2026-04-03T22:40:32.737939",
        "status": "PENDING"
    },
    {
        "id": 2,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 3,
        "totalPrice": 3600.0,
        "orderDate": "2026-04-03T23:19:07.307864",
        "status": "PENDING"
    },
    {
        "id": 3,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 3,
        "totalPrice": 3600.0,
        "orderDate": "2026-04-03T23:20:09.522337",
        "status": "PENDING"
    }
]
```


Exercice 3 : Gestion des erreurs

1 –

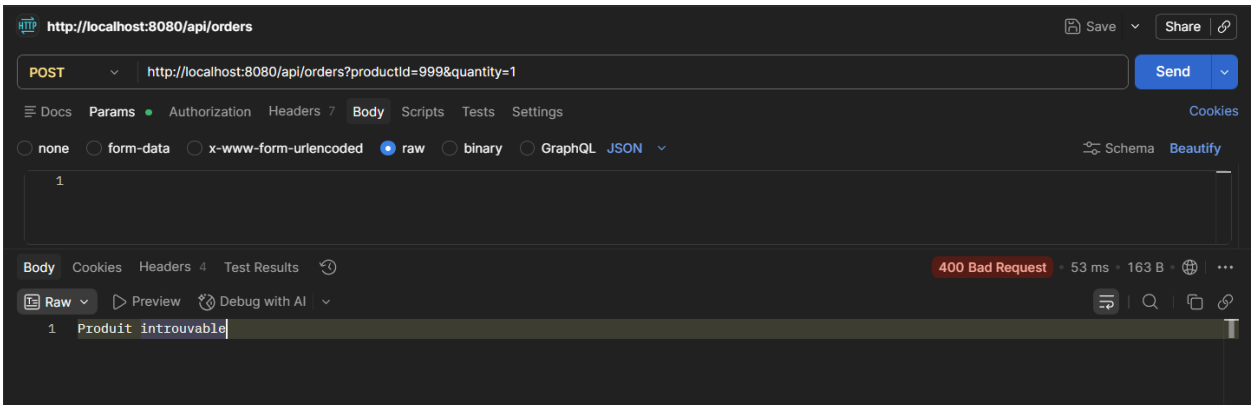
Project

- TP-4 C:\Users\ASUS\Desktop\TP-4
  - .idea
  - API-service
  - eureka-server
  - order-server
    - .idea
    - .mvn
    - src
      - main
        - java
          - com.SOA.order_server
            - client
            - Controller
            - dto
            - model
            - Repository
            - Service
              - OrderService
              - OrderServerApplication
        - resources
      - test
    - target
    - .gitattributes
    - .gitignore
    - HELP.md
    - mvnw
    - mvnw.cmd
    - pom.xml
  - product-server
- External Libraries
- Scratches and Consoles

\application.properties | order-server\...\application.properties | product-server\...\application.properties | OrderService.java

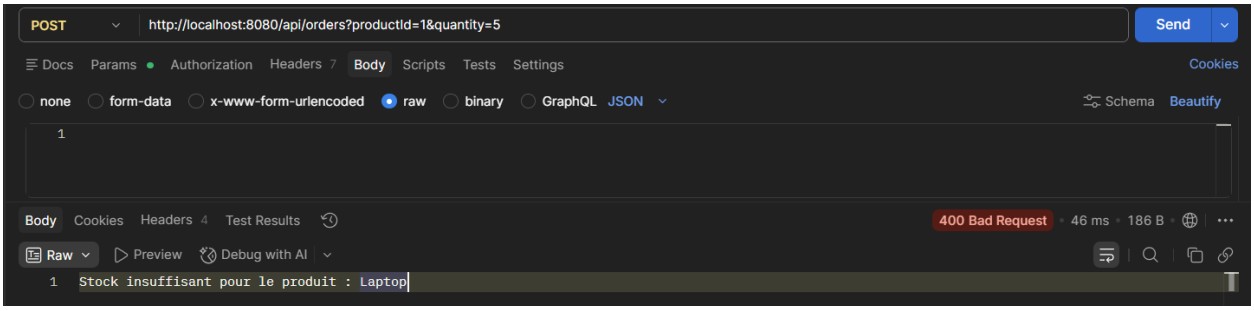
```
15  public class OrderService {
20      }
21      public Order createOrder(Long productId, Integer quantity) {  1 usage

23          try {
24              ProductDTO product = productClient.getProductById(productId);
25
26              if (product == null) {
27                  throw new RuntimeException("Produit introuvable");
28              }
29
30              if (product.getStock() < quantity) {
31                  throw new RuntimeException("Stock insuffisant pour le produit : " + product.getName());
32              }
33
34              Order order = new Order();
35              order.setProductId(productId);
36              order.setProductName(product.getName());
37              order.setQuantity(quantity);
38              order.setTotalPrice(product.getPrice() * quantity);
39              order.setOrderDate(LocalDateTime.now());
40              order.setStatus("PENDING");
41
42              Order savedOrder = orderRepository.save(order);
43
44              productClient.updateStock(productId, quantity);
45
46              return savedOrder;
47
48          } catch (FeignException.NotFound e) {
49              throw new RuntimeException("Produit introuvable");
50          } catch (RetryableException e) {
51              throw new RuntimeException("Service produit indisponible");
52          } catch (FeignException e) {
53              throw new RuntimeException("Erreur du service produit : " + e.getMessage());
54          }
```

2 – Vérification via Postman

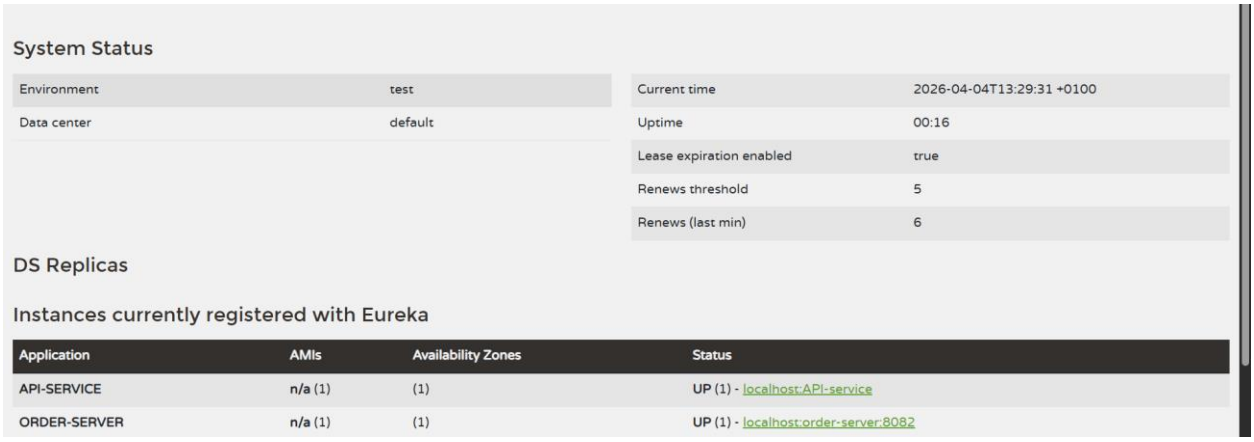


Le produit avec le Id = 999 n`existe pas alors le message de gestion d`erreur apparaisse.

3 – Stock insuffisant



4 – On arrête le service produit



Fin du rapport